

Creating a Gradle Project in IntelliJ IDEA

Learn how to set up a Gradle-based Java project for unit testing with JUnit 6

 by Rodrigo Martins Pagliares

Last update: 11/25/2025



Getting Started

Configuring Gradle Project Coordinates



Expand Artifact Coordinates

Define project metadata to uniquely identify your project



Group ID

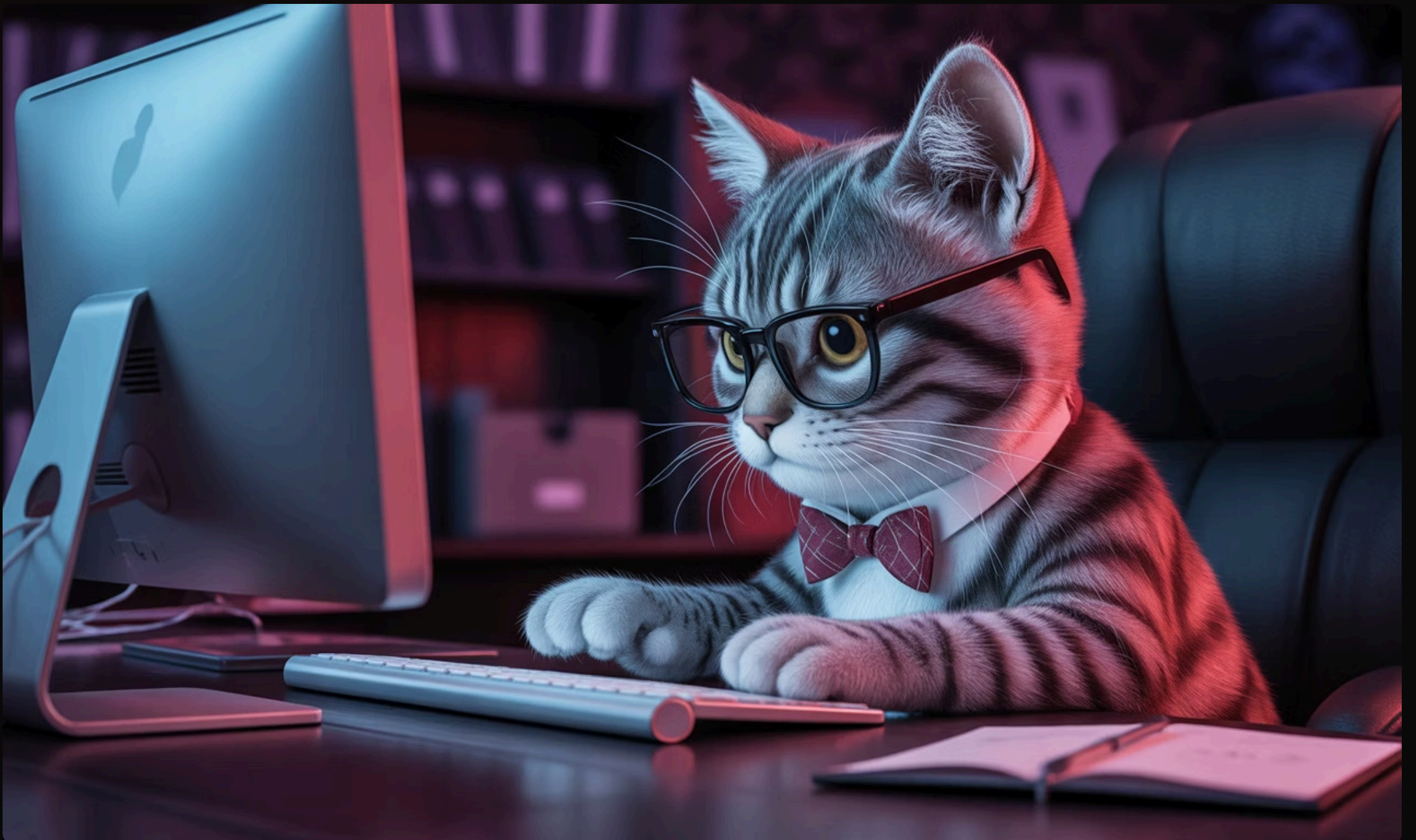
Group ID usually reflects the reversed domain name of your organization

- Example: `org.example`



Artifact ID

Artifact ID is typically the name of the project





Getting Started

Creating a Gradle Project in IntelliJ IDEA

Gradle setup lessons can be **skipped** if you don't use **Gradle**

1. **New project** → **Project name** (CalculatorGradleProject) → **Location** (Target folder)
2. **Artifact Coordinates:**

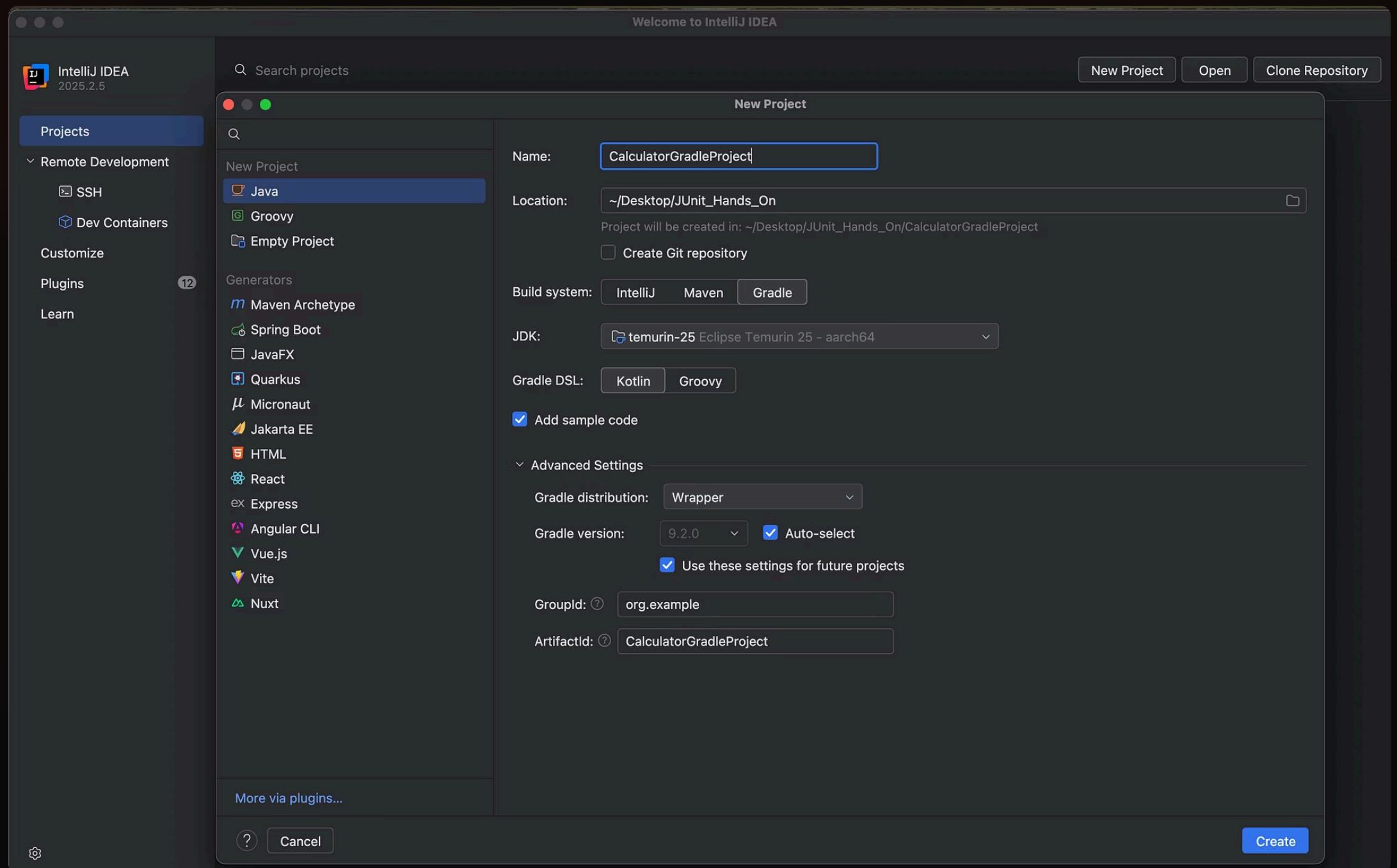
`groupid = org.example`

`artifactid = CalculatorGradleProject`

3. **Finish** (Generate the **Gradle** project structure)

Getting Started

Creating a Gradle Project in IntelliJ IDEA

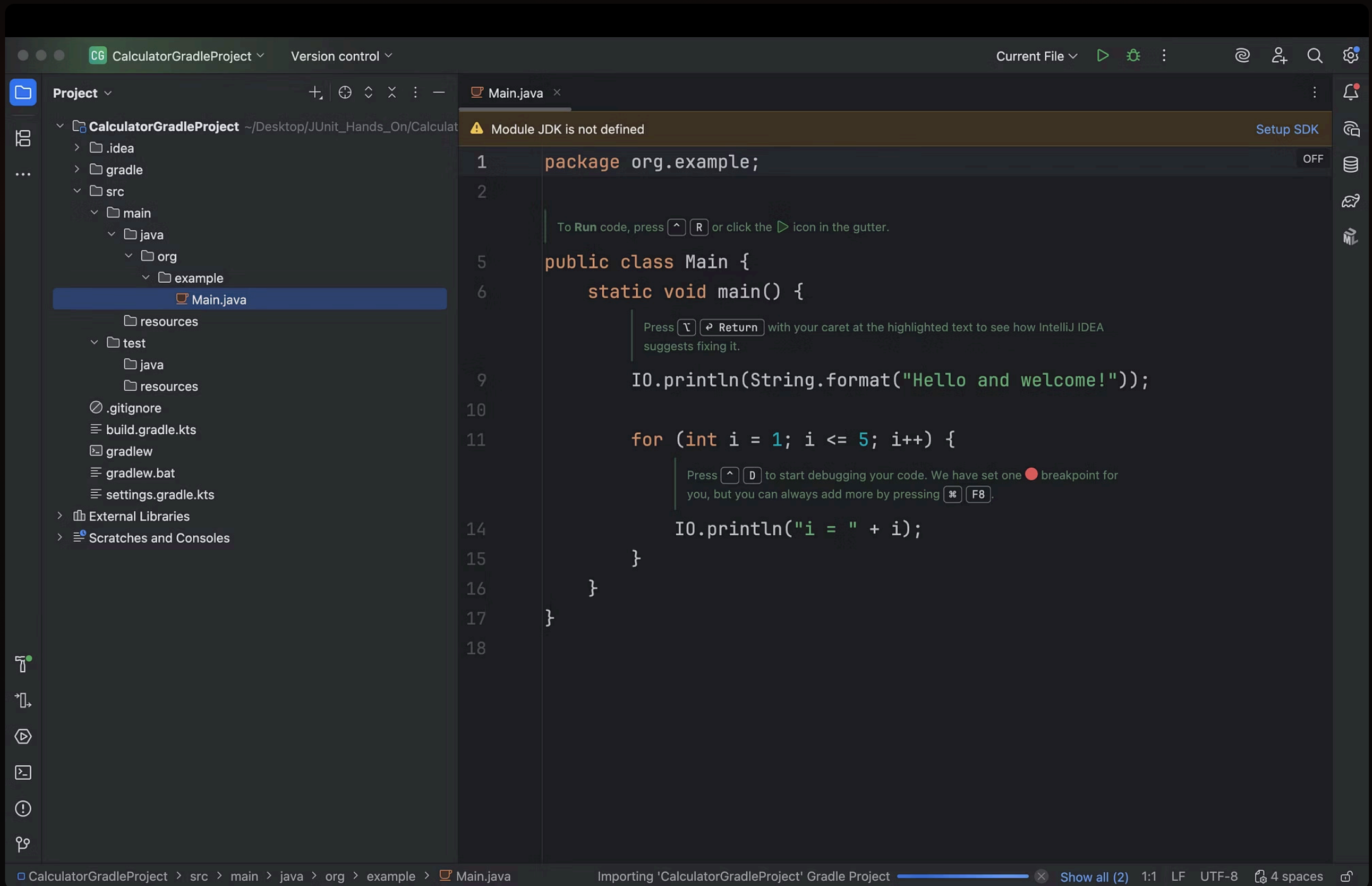


New project → **Project name** (CalculatorGradleProject) → **Location** (Target folder)

Click **Create** to create the **Gradle project structure**.

Getting Started

Project created!



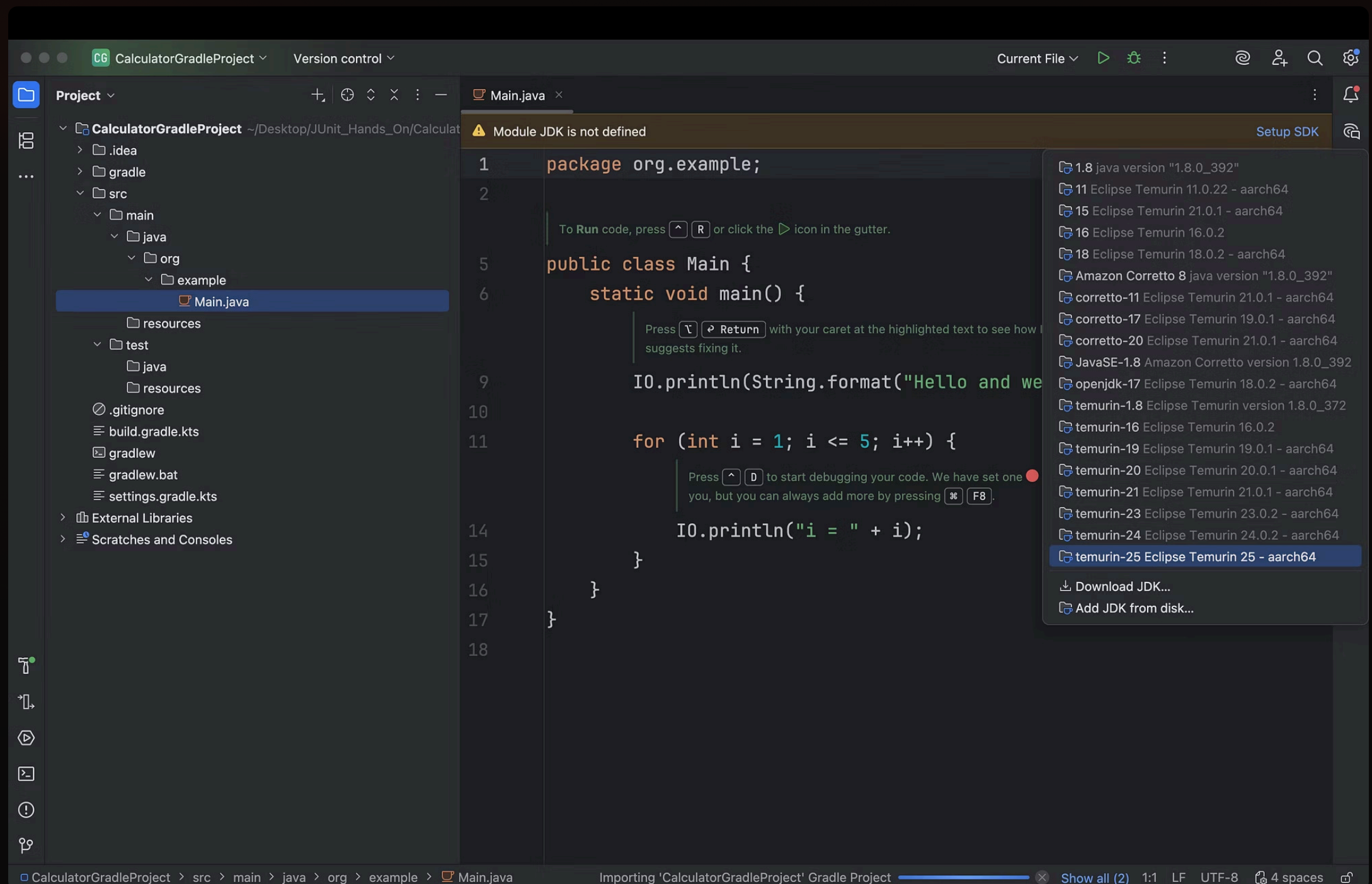
But notice that in **my local environment** the **Module JDK is not defined** (See next slide)

Getting Started

Setup JDK

You can skip this step if you did not receive the warning "**Module JDK is not defined**".

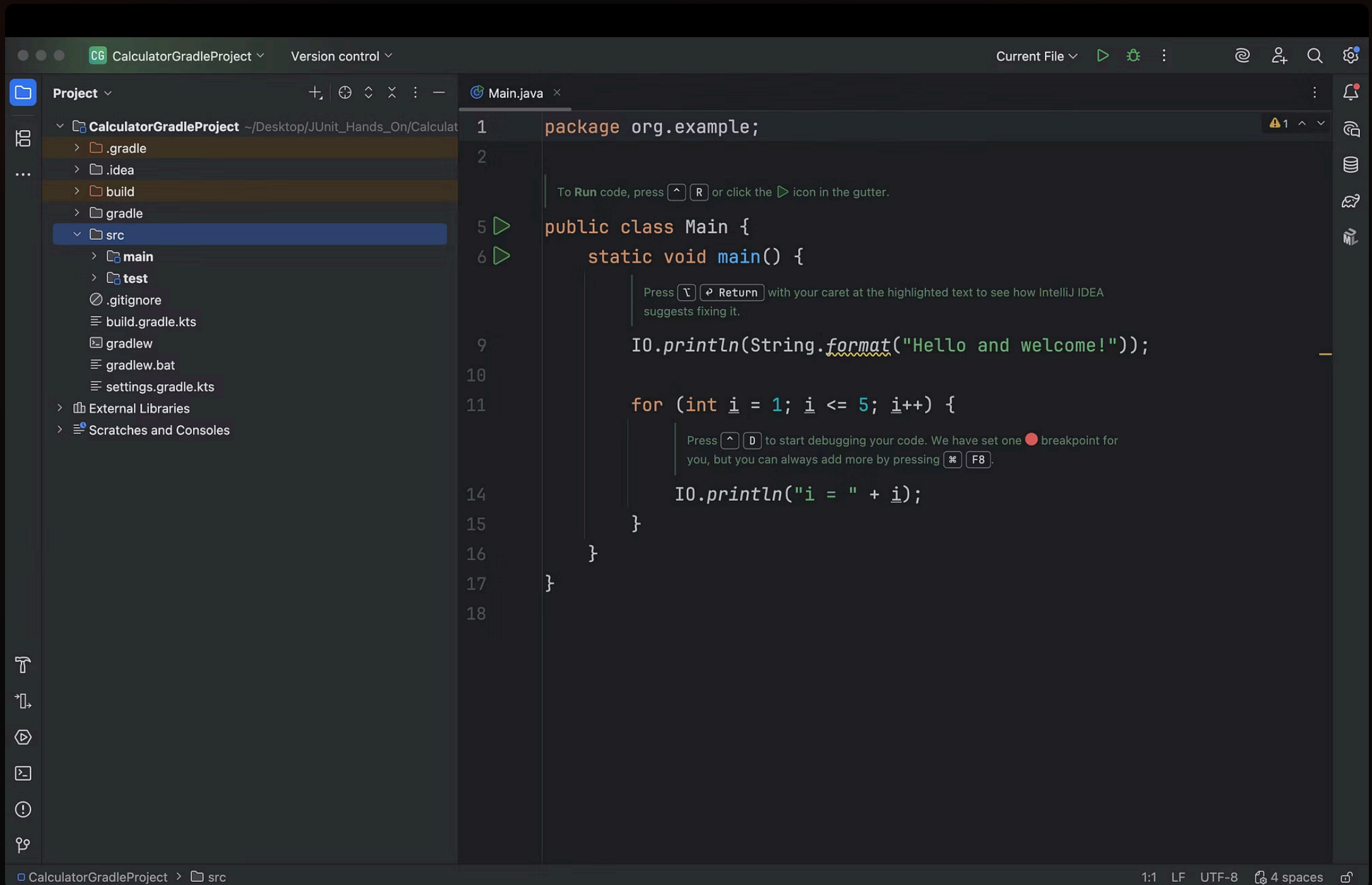
In my case, I selected my default JDK (Temurin-25 Eclipse Temurin 25 - aarch64)



Getting Started

Setup JDK

No more warning message!



Getting Started

Gradle Project Structure Overview

The Gradle project includes the standard folder layout:

- `src/main/java` for application code
- `src/test/java` for test classes

The `build.gradle.kts` file manages **dependencies** and **plugins**

JUnit 6 is supported through **individual** or **aggregate Gradle dependencies**

Future lessons will show how to add a **single aggregate JUnit dependency**

The structure prepares your project for adding and running tests

Conclusion

1

IntelliJ simplifies the creation of Gradle-based Java projects

2

Group and artifact identifiers uniquely define each project

3

Gradle's structure mirrors Maven's separation of main and test sources

4

Upcoming lessons add JUnit support using the build.gradle file

Open Floor Discussion



Do you prefer **Maven** or **Gradle** for dependency management?



Have you customized your `build.gradle` before for testing frameworks?



Which project structure (**Maven** or **Gradle**) feels more intuitive to you?